

安全漏洞审计报告 · 第1轮

用户信息管理平台 密码安全漏洞审计报告

User Management Platform — Password Security Assessment

目标系统：用户信息管理平台 (Flask)

报告日期：2026-07-07

漏洞等级：严重 × 4 | 中危 × 4

修复状态：✅ 全部已修复

目录

1. 报告概述
2. 系统架构
3. 漏洞详情 — 8 项密码安全漏洞
4. 修复方案
5. 修复前后对照
6. 修复验证
7. 修复总结

1. 报告概述

1.1 项目背景

本报告针对一个基于 Python Flask 框架构建的简易用户信息管理平台进行安全审计。该系统提供用户登录、信息展示、登出等基本功能。平台初始版本故意引入了多项常见 **Web** 安全漏洞，旨在作为安全教学示例，演示从"漏洞引入"到"漏洞修复"的完整流程。

1.2 审计范围

- 审计目标：用户登录模块的密码安全性
- 审计方法：代码审计 + 黑盒测试
- 涉及文件：app.py、templates/login.html、templates/index.html

1.3 风险等级定义

等级	说明
 严重	可直接导致密码泄露、未授权登录或会话劫持，需立即修复
 中危	可辅助攻击或增加攻击面，应在合理时间内修复

2. 系统架构

2.1 技术栈

组件	技术
后端框架	Python Flask
模板引擎	Jinja2
用户存储	内存字典 (开发演示用)
会话管理	Flask Signed Cookies

2.2 功能模块

路由	方法	功能
/	GET	首页，展示当前登录用户完整信息
/login	GET / POST	用户登录，提交用户名和密码
/logout	GET	清除会话并跳转回首页

3. 漏洞详情

本轮审计共发现 8 项密码安全漏洞，其中严重等级 4 项，中等级 4 项。

4

严重漏洞

4

中危漏洞

8/8

已修复

VULN-01：密码明文存储 位置：app.py — USERS 字典 描述：密码以明文形式直接硬编码在 Python 源代码中："password": "admin123"。一旦代码被泄露，所有用户密码直接暴露。 攻击场景：获取 app.py → 直接读取明文密码 → 无需破解即可登录。 严重
VULN-02：密码明文展示在前端 位置：templates/index.html — {{ user.password }} 描述：登录成功后密码原文渲染到 HTML 页面，任何看到屏幕或通过 XSS 攻击的人都能获取密码。 严重
VULN-03：明文密码比对 位置：app.py — login 路由 描述：使用 == 直接比对密码字符串，数据库泄露后可立即使用密码原文登录。 严重
VULN-04：HTML 注释泄露默认密码 位置：templates/login.html — 第 1 行 描述：登录页 HTML 源码中包含：<!-- 调试信息 - 默认管理员账号 用户名：admin 密码：admin123 -->。查看页面源码即可获取管理员密码。 严重
VULN-05：页面明文提示默认账号 位置：templates/login.html 描述：登录页底部直接展示"默认账号：admin / admin123"，降低攻击者信息收集成本。 中危

VULN-06 : Secret Key 过弱且硬编码

● 严重

位置：app.py — app.secret_key = "dev-key-2025"

描述：密钥为固定弱字符串，攻击者知道后可以伪造任意用户 session 以登录任意账号。

VULN-07 : 无登录频率限制

● 中危

位置：app.py — login 路由

描述：登录接口无验证码、无失败计数、无 IP 封禁，可无限次暴力破解密码。

VULN-08 : Debug 模式开启

● 中危

位置：app.py — app.run(debug=True)

描述：出错时显示详细 Python 调试页面，暴露源码、变量和环境信息。

4. 修复方案

VULN-01 / VULN-03 : 密码哈希存储与比对

使用 `werkzeug.security` 进行 `bcrypt` 风格哈希处理。

```
from werkzeug.security import generate_password_hash, check_password_hash

# 存储
"password": generate_password_hash("admin123")

# 比对
check_password_hash(USERS[username]["password"], password)
```

VULN-02 : 删除前端密码展示

从 `index.html` 中删除 `{{ user.password }}` 模板变量。

VULN-04 / VULN-05 : 清除页面密码泄露

删除 HTML 调试注释和默认账号提示段落。

VULN-06 : Secret Key 加固

```
app.secret_key = os.environ.get("SECRET_KEY", os.urandom(24).hex())
```

VULN-07 : IP 级别登录限流

同一 IP 连续 5 次登录失败后封禁 5 分钟，自动解封。

VULN-08 : Debug 模式环境变量控制

```
debug_mode = os.environ.get("FLASK_DEBUG", "false").lower() == "true"
```

5. 修复前后对照

5.1 app.py — 核心逻辑

✖ 修复前

```
app.secret_key = "dev-key-2025"

USERS = { "admin": {
    "password": "admin123",
} }

USERS[username]["password"] == password

app.run(debug=True)
```

✔ 修复后

```
app.secret_key = os.environ.get(...)

USERS = { "admin": {
    "password":
        generate_password_hash("admin123"),
} }

check_password_hash(...)

app.run(debug=False)
```

5.2 templates — 登录页 & 首页

✖ 修复前

login.html: HTML注释泄露密码
login.html: 页面显示默认账号
index.html: 显示 {{ user.password }}

✔ 修复后

注释已删除
提示已删除
密码字段已删除

6. 修复验证

#	测试用例	结果
1	查看登录页源码，搜索 "admin123"	✓ 通过
2	正确密码登录 (admin / admin123)	✓ 通过
3	首页不显示密码字段	✓ 通过
4	错误密码提示	✓ 通过
5	连续 5 次失败后封禁	✓ 通过
6	secret_key 非固定值 "dev-key-2025"	✓ 通过
7	debug 模式默认关闭	✓ 通过
8	数据库无明文密码	✓ 通过

7. 修复总结

#	漏洞名称	原等级	状态
1	密码明文存储	 严重	 已修复
2	密码明文前端展示	 严重	 已修复
3	明文密码比对	 严重	 已修复
4	HTML 注释泄密	 严重	 已修复
5	页面提示默认密码	 中危	 已修复
6	Secret Key 过弱	 严重	 已修复
7	无登录限流	 中危	 已修复
8	Debug 模式	 中危	 已修复

安全建议

- 生产环境应使用数据库替代内存字典存储
- 部署时使用 HTTPS 加密传输
- 增加密码强度校验规则
- 集成 CAPTCHA 验证码加强暴力破解防御
- 记录所有登录尝试到日志文件便于追溯